Traffic Shaping and Load Shedding

reject some requests on purpose, so the important ones survive

Watch the 4-min video

In one line: under overload, queues grow, clients time out and retry, and useful throughput collapses; load shedding rejects excess work early and cheaply (a fast 503), drops low-priority traffic first, caps in-flight requests and queues with admission control, and clients cooperate with backoff and retry budgets.

1 Why overload is so dangerous

It's Black Friday. Traffic is five times normal and everything slows down together: checkout, search, analytics beacons. Soon nothing works at all. Why does it get *worse* than just slow?

When a server gets more work than it can do, requests wait in queues. Latency grows, clients time out and **retry**, and the retries add even more load. The server spends its time on requests whose clients have already given up. The CPU is at 100%, but **goodput** (requests completed in time, that someone still wants) collapses.

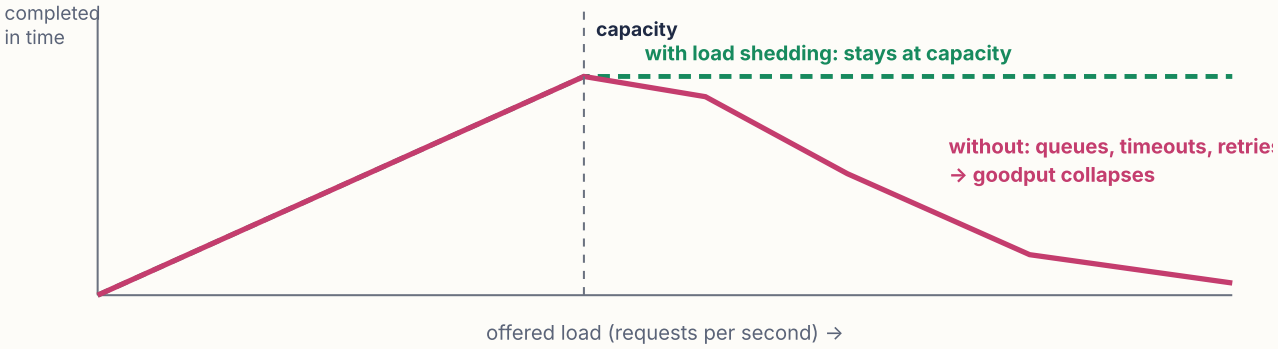


Figure 1. Beyond capacity, an unprotected server completes fewer and fewer requests in time; with load shedding it keeps serving at capacity and rejects the rest.

2 Load shedding: fail fast, on purpose

Load shedding means rejecting excess work early and cheaply, with a fast error such as **503 Service Unavailable** (with a `Retry-After` header), instead of accepting it and doing it badly. A quick rejection costs about a millisecond; the client can back off or try elsewhere; and the requests you do accept finish quickly. It feels wrong to refuse users, but it's the difference between serving 100% of capacity and serving almost nobody.

3 Not all traffic is equal

Give requests a **priority**. Under pressure, drop the sheddable tier first, then the important tier, and protect critical traffic to the end. The gateway can tag each request (by route, client or a header) so every service downstream applies the same policy.

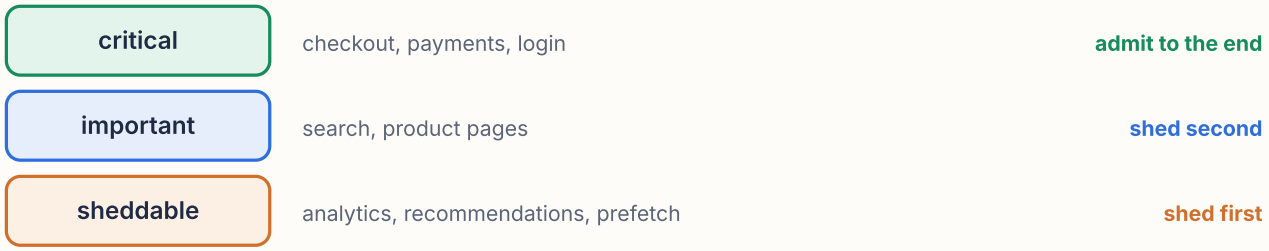


Figure 2. Three priority tiers. Shedding starts at the bottom; critical requests are admitted as long as possible.

4 Knowing you're overloaded: admission control

Admission control checks before accepting work: cap the number of **in-flight** requests and the **queue size**, so requests never wait forever. A request that would wait longer than its client's timeout is better rejected now.

Adaptive concurrency limits go further: they watch latency and shrink the allowed concurrency when latency rises, and grow it when latency is fine, like TCP congestion control. Netflix's *concurrency-limits* library and Envoy's adaptive concurrency filter work this way, so you don't have to guess the right limit.

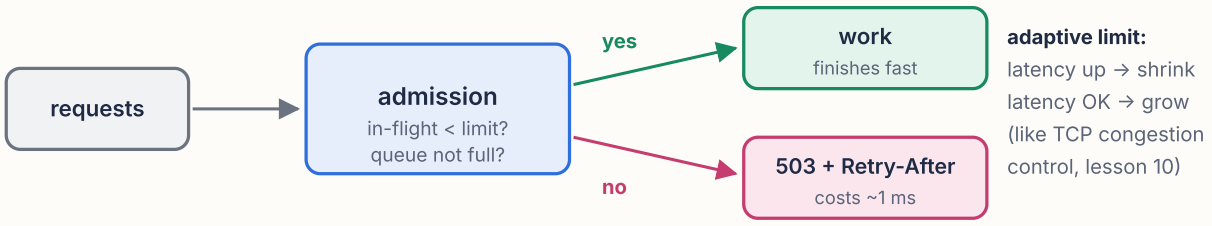


Figure 3. Admission control: work is accepted only if there's room; otherwise the request gets a cheap, immediate 503.

WORKED EXAMPLE: BLACK FRIDAY

CPU on the API fleet crosses 80%. The gateway starts dropping analytics and tracking requests with a 503. Recommendations fall back to a cached "popular items" list. Checkout requests carry a `priority: critical` header and are always admitted. Revenue keeps flowing, latency on checkout stays normal, and nobody notices the missing analytics events.

5 Shaping before overload

- **Rate limits and quotas** cap each client, so one integration can't eat everyone's capacity (Phase 2, API lesson 12).
- **Queues** smooth bursts, so background jobs run at a steady pace instead of spiking.
- **Cooperative clients** honour `Retry-After`, use exponential backoff with jitter, and keep a retry budget, so they don't hammer a struggling service.

COMMON MISTAKES

- Unbounded queues: they hide overload until every request is late.
- Shedding at the database instead of the edge, after the expensive work is done.
- Treating all requests the same, so checkout fails alongside analytics.
- Clients that retry 503s immediately and forever.

6 Key terms

Overload: more offered work than capacity

Load shedding: rejecting excess work early and cheaply

Admission control: deciding whether to accept a request at all

Retry-After: header telling clients when to come back

Goodput: useful requests completed in time

Priority tier: critical, important, sheddable

Adaptive concurrency: a limit that follows latency

Retry budget: cap on retries as a share of traffic

7 Interview questions

Q1. How do you keep a system alive under overload? BASIC

Shed load early with fast rejections, prioritise critical traffic and drop low-priority tiers first, cap in-flight requests and queue sizes, and use adaptive concurrency limits, while clients back off with jitter.

Q2. Why not just accept everything? BASIC

Because queues grow, requests time out, retries add load, and throughput collapses for everyone; the server ends up doing work nobody is waiting for.

Q3. Design load shedding for an e-commerce API on Black Friday. SCENARIO

Tag requests by priority at the gateway (checkout critical, browse important, analytics sheddable), shed from the bottom when CPU or latency crosses thresholds, serve cached fallbacks for recommendations, cap queues, and return 503 with Retry-After.

Q4. Where should load shedding happen? SCENARIO

As early as possible, at the gateway or the service's front door, before expensive work; and in every service, since each one can be overloaded independently.

Q5. Why is an unbounded queue a bad idea? TRICKY

It converts overload into latency: every request waits longer and longer, most time out, and the server still processes them. A bounded queue plus fast rejection keeps latency low for the requests it does accept.

Q6. How do adaptive concurrency limits work? TRICKY

They measure latency and adjust the allowed number of in-flight requests, like TCP congestion control: if latency rises above the baseline, the limit shrinks; if it stays low, the limit grows. No fixed number to guess.

Next: SD-G-011 · Global Server Load Balancing and Regional Routing backendroadmap.com